

LE BASI DI REACT

Come creare un React App

Definizione. `create-react-app` è uno **starter kit**: uno script che genera un progetto React già configurato, con compilatore (Babel), bundler (Webpack), dev server e struttura di cartelle pronti.

Perché serve. Il codice React che scrivi non è JavaScript che il browser capisce. JSX (lo vediamo tra poco) va tradotto. Configurare a mano Babel e Webpack sono ore di lavoro. `create-react-app` fa tutto questo al posto tuo.

```
npx create-react-app prima-app
```

Al termine dell'installazione ti vengono suggeriti quattro script. La trascrizione li elenca tutti, ecco cosa fanno davvero:

Comando	Funzione
<code>npm start</code>	Avvia il development server , di norma su <code>localhost:3000</code> . Ha l'hot reload: salvi un file e il browser si aggiorna da solo.
<code>npm run build</code>	Crea la build statica ottimizzata (minificata) nella cartella <code>build/</code> , pronta per la pubblicazione online.
<code>npm test</code>	Lancia i test automatici. Il corso dichiara esplicitamente che non li tratta.
<code>npm run eject</code>	Irreversibile. Estrae la configurazione nascosta di Webpack per poterla modificare a mano. Il corso dice chiaramente di non usarlo.

Nota per il 2026: `create-react-app` non è più il metodo raccomandato — è stato deprecato. Oggi si usa **Vite**:

```
npm create vite@latest prima-app -- --template react
```

La logica di React insegnata nel corso resta identica; cambia solo lo strumento che genera il progetto. È il tipo di disallineamento normale in un corso registrato qualche anno fa.

Esplorazione della prima React App

Definizione. Capire a cosa serve ogni file generato, per sapere dove mettere le mani.

```
prima-app/
├── node_modules/  ← tutte le librerie installate (non si tocca mai)
├── public/
│   └── index.html  ← l'UNICA pagina HTML dell'applicazione
├── src/
│   ├── index.js    ← punto di ingresso: "inietta" React nella pagina
│   ├── App.js      ← il componente principale
│   └── App.css      ← stili
```

```
|   └─ index.css
|   └─ package.json    ← nome, dipendenze, script
|   └─ .gitignore       ← file da NON caricare su GitHub
```

public/index.html. È un normalissimo file HTML — la trascrizione lo ripete tre volte perché è un punto che confonde. Puoi metterci meta tag, Google Fonts, cambiare il `<title>`. La riga cruciale è una sola:

```
<div id="root"></div>
```

Questo div vuoto è il contenitore. Tutta la tua applicazione verrà inserita lì dentro da JavaScript.

src/index.js. È il file che fa l'inserimento (nella trascrizione: "inietta l'applicazione all'interno dell'elemento di root"):

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(<App />);
```

Traduzione a parole: *prendi l'elemento con id "root" nell'HTML, e disegnaci dentro il componente App.*

Questo spiega cosa significa Single Page Application (SPA). Esiste una sola pagina HTML. Quando navighi tra le sezioni del sito, il browser non ricarica nulla: è JavaScript che sostituisce il contenuto del div `root`. Da qui la sensazione di istantaneità.

package.json. Il documento d'identità del progetto. Contiene le `dependencies` (le librerie) e gli `scripts`. È il file che permette a chiunque di ricostruire il progetto con un solo `npm install`.

.gitignore. Elenca i file da escludere da Git. Due voci fondamentali: `node_modules` (pesantissima e ricostruibile) e `.env` (che contiene chiavi API segrete — la trascrizione insiste molto su questo punto).

Pulizia iniziale. Il corso elimina `logo.svg`, `App.test.js`, `reportWebVitals.js`, `setupTests.js` e ripulisce gli import relativi. Attenzione: se cancelli un file ma resta il suo import, l'applicazione va in errore. È il primo errore che vedrai.

Creare un componente

Definizione. Un componente React è una **funzione JavaScript che ritorna del markup JSX**. È l'unità base di costruzione dell'interfaccia.

Le due regole assolute (la trascrizione le chiama "regole che vanno strettamente rispettate"):

1. **Il nome deve iniziare con la lettera maiuscola.** Card, non card. React usa questa convenzione per distinguere i tuoi componenti (`<Card />`) dai tag HTML nativi (`<div>`). Se scrivi minuscolo, React cerca un tag HTML chiamato "card", non lo trova, e non renderizza nulla.
2. **Deve ritornare qualcosa.** JSX, oppure `null` se non vuoi mostrare niente.

```
// src/Componente.js
const Componente = () => {
  return <h1>Ciao, sono un componente</h1>;
};

export default Componente;
```

Import ed export. Un file espone i suoi contenuti con `export` e li riceve con `import`.

`export default` — uno per file. Chi importa può dare il nome che vuole:

```
export default Componente;

// altrove:
import QualsiasiNome from './Componente'; // funziona
```

Questo è ciò che la trascrizione intende con "ci permette di rinominare il file come più ci pare".

`export` nominale — quanti ne vuoi, ma il nome deve corrispondere esattamente:

```
export const Bottone = () => <button>Click</button>;
export const Titolo = () => <h1>Titolo</h1>;

// altrove:
import { Bottone, Titolo } from './Elementi';
```

Lo snippet rfc. Con l'estensione ES7 React snippets installata, scrivi `rfc` in un file vuoto, premi Tab, e ottieni lo scheletro completo del componente già con `import` ed `export default`. Per questo il corso la consiglia.

Le regole della sintassi JSX

Definizione. JSX sta per *JavaScript XML*. È un'estensione della sintassi di JavaScript che ti permette di scrivere qualcosa che assomiglia all'HTML dentro il codice JavaScript. Non è HTML e non è una stringa: viene **compilato** in chiamate a funzione.

Quello che scrivi tu:

```
const elemento = <h1 className="titolo">Ciao</h1>;
```

Quello che il browser riceve davvero:

```
const elemento = React.createElement("h1", { className: "titolo" }, "Ciao");
```

Capire questo spiega tutte le regole che seguono, perché sono conseguenze di JavaScript, non capricci.

Regola 1 — Un solo elemento radice. Una funzione può ritornare un solo valore, quindi JSX deve avere un contenitore unico.

```
// ERRORE

return (
  <h1>Titolo</h1>
  <p>Paragrafo</p>
);

// CORRETTO

return (
  <div>
    <h1>Titolo</h1>
    <p>Paragrafo</p>
  </div>
);

// MEGLIO: React.Fragment, che non crea un div inutile nel DOM

return (
  <>
    <h1>Titolo</h1>
    <p>Paragrafo</p>
  </>
);
```

Regola 2 — Tutti i tag vanno chiusi. Anche quelli che in HTML sono auto-chiudenti:

```

<input type="text" />
<br />
```

Regola 3 — className al posto di class. Perché class è una parola riservata di JavaScript.

```
<div className="card">...</div>
```

Regola 4 — camelCase per gli attributi. onclick → onClick, tabIndex → tabIndex, for → htmlFor.

Regola 5 — Lo stile inline è un oggetto, e le proprietà CSS in camelCase:

```
// HTML: <div style="background-color: red; font-size: 20px">
<div style={{ backgroundColor: "red", fontSize: "20px" }}>
```

Le doppie graffe non sono una stranezza: la prima coppia significa "qui entro in JavaScript", la seconda è l'oggetto vero e proprio.

Regola 6 — I commenti si scrivono così:

```
{/* questo è un commento in JSX */}
```

Componenti innestati

Definizione. Inserire un componente dentro un altro. È il meccanismo con cui si costruisce l'albero dell'interfaccia.

```
const Header = () => <header><h1>Il mio sito</h1></header>;
const Footer = () => <footer><p>© 2026</p></footer>;
```

```
const App = () => {
  return (
    <div>
      <Header />
      <main>Contenuto principale</main>
      <Footer />
    </div>
  );
};
```

Si genera un **albero dei componenti**: App è il genitore (parent), Header e Footer sono i figli (children). Questa gerarchia diventerà centrale più avanti, perché i dati scendono dall'alto verso il basso lungo questo albero.

Utilizzare JavaScript in JSX

Definizione. Le parentesi graffe {} sono la porta di ingresso: dentro JSX, tutto ciò che sta fra graffe viene valutato come espressione JavaScript.

```
const App = () => {
  const nome = "Dario";
  const prezzo = 19.9;
  const utente = { città: "Barletta" };

  return (
    <div>
      <h1>Ciao {nome}</h1>      {/* variabile */}
      <p>Totale: {prezzo * 2} €</p>  {/* operazione */}
      <p>Città: {utente.città}</p>  {/* proprietà di oggetto */}
      <p>{nome.toUpperCase()}</p>  {/* chiamata a metodo */}
      <p>{new Date().getFullYear()}</p>  {/* espressione qualsiasi */}
    </div>
  );
};
```

Il vincolo da capire: dentro le graffe vanno solo espressioni, non istruzioni. Un'espressione produce un valore; un'istruzione esegue un'azione. `if` e `for` sono istruzioni, quindi non si possono mettere dentro le graffe.

```
{ if (loggato) { ... } }      // ERRORE
{ loggato ? <Profilo /> : <Login /> } // CORRETTO: l'operatore ternario è un'espressione
```

Questo è il motivo per cui in React si usano ovunque l'operatore ternario e `&&` invece di `if`. Non è uno stile: è una necessità strutturale.

Il Props Object

Definizione. Le **props** (abbreviazione di *properties*) sono i dati che un componente genitore passa a un componente figlio. Tecnicamente sono **un unico oggetto** passato come primo parametro della funzione componente.

Questo è il passaggio concettuale centrale della lezione, e la trascrizione lo costruisce bene: *"un componente altro non è che una funzione, e come tutte le funzioni accetta dei parametri"*.

```
// Il componente riceve l'oggetto props
const Prodotto = (props) => {
  console.log(props); // { nome: "Dario", zona: "arancione" }
  return <h2>{props.nome}</h2>;
};
```

```
};

// Il genitore passa i valori come se fossero attributi HTML
const App = () => {
  return <Prodotto nome="Dario" zona="arancione" />;
};
```

Tre osservazioni importanti dalla trascrizione:

1. Ogni istanza del componente ha le sue props. Se scrivi cinque `<Prodotto />` e fai `console.log(props)`, vedi cinque oggetti separati. *"Ogni componente ha una logica indipendente"*: non condividono nulla.

2. Per passare un numero (o un booleano, o un array) servono le graffe.

```
<Prodotto numero="2" />    // passa la STRINGA "2"
<Prodotto numero={2} />    // passa il NUMERO 2
<Prodotto attivo={true} />
<Prodotto lista={[1, 2, 3]} />
```

Senza graffe passi sempre una stringa, e poi ti ritrovi con `"2" + 1 = "21"`.

3. Le props sono in sola lettura. Un componente non deve mai modificare le props che riceve. Se un dato deve cambiare, si usa lo *state* (Lezione 3).

Esempio completo, quello sviluppato nel corso — la riusabilità.

```
// I dati
const primaCard = {
  nome: "Cuffie Bluetooth",
  prezzo: 49.9,
  img: "cuffie.jpg",
  costoFinale: 59.9
};

const secondaCard = {
  nome: "Mouse Wireless",
  prezzo: 19.9,
  img: "mouse.jpg",
  costoFinale: 24.9
};
```

```
// UN SOLO componente
const Prodotto = (props) => {
  return (
    <article className="card">
      <img src={props.img} alt={props.nome} />
      <h3>{props.nome}</h3>
      <p>{props.prezzo} €</p>
      <button>Compra a {props.costoFinale} €</button>
    </article>
  );
};

// Usato due volte con dati diversi
const App = () => (
  <div>
    <Prodotto nome={primaCard.nome} prezzo={primaCard.prezzo}
      img={primaCard.img} costoFinale={primaCard.costoFinale} />
    <Prodotto nome={secondaCard.nome} prezzo={secondaCard.prezzo}
      img={secondaCard.img} costoFinale={secondaCard.costoFinale} />
  </div>
);
```

Scrivi la card una volta sola e la riutilizzi con dati qualsiasi. Come dice la trascrizione: se avessi *"un array con 50 di queste card"* potresti costruire la pagina di uno shop intero. Questo è il valore concreto della riusabilità.

Decostruzione oggetto e Spread Operator

Il problema. Nell'esempio sopra, ogni componente occupa quattro righe di props ripetitive. Con dieci proprietà diventa illeggibile e difficile da mantenere.

Spread operator (. . .) — per PASSARE le props in modo compatto.

Definizione. L'operatore . . . espande il contenuto di un oggetto o di un array nei suoi singoli elementi.

// Da questo:

```
<Prodotto nome={primaCard.nome} prezzo={primaCard.prezzo}
  img={primaCard.img} costoFinale={primaCard.costoFinale} />
```

// A questo, che è esattamente equivalente:

```
<Prodotto {...primaCard} />
```

Le graffe esterne servono per entrare in JavaScript; i tre punti espandono l'oggetto in coppie chiave-valore. Il risultato compilato è identico, ma il codice è *"molto più ordinato e di più facile lettura"*.

L'esempio di supporto usato nel corso per far capire l'operatore, fuori da React:

```
const numeri = [1, 2, 3, 4, 5, 6, 7, 8, 9];
```

```
Math.max(numeri);    // NaN — Math.max vuole numeri singoli, non un array
```

```
Math.max(...numeri); // 9 — lo spread li espande uno per uno
```

Destrutturazione — per ACCEDERE alle props in modo pulito.

Definizione. La destrutturazione è l'operazione complementare allo spread: estrae valori singoli da un oggetto o array e li assegna a variabili.

```
const utente = { nome: "Dario", eta: 30, citta: "Barletta" };
```

```
const { nome, citta } = utente;
```

```
// nome = "Dario", citta = "Barletta"
```

Attenzione a un dettaglio che la trascrizione chiarisce esplicitamente: **queste graffe non hanno nulla a che vedere con quelle di JSX**. Qui sei già dentro JavaScript puro; è la sintassi standard della destrutturazione. E destrutturare **non distrugge** l'oggetto originale: lo copia soltanto.

Applicata alle props, esistono due livelli.

Livello 1 — dentro il corpo della funzione:

```
const Prodotto = (props) => {
  const { nome, prezzo, img, costoFinale } = props;
  return (
    <article>
      <img src={img} alt={nome} />
      <h3>{nome}</h3>
      <p>{prezzo} €</p>
      <button>Compra a {costoFinale} €</button>
    </article>
  );
};
```

```
};
```

Livello 2 — direttamente nei parametri, ed è la forma che vedrai ovunque nel codice React reale:

```
const Prodotto = ({ nome, prezzo, img, costoFinale }) => {  
  return (  
    <article>  
      <img src={img} alt={nome} />  
      <h3>{nome}</h3>  
      <p>{prezzo} €</p>  
      <button>Compra a {costoFinale} €</button>  
    </article>  
  );  
};
```

Guarda la firma della funzione: ti dice immediatamente quali dati quel componente si aspetta. È documentazione gratuita.

Riassunto della coppia: spread e destrutturazione sono operazioni complementari. Lo spread *impacchetta* l'oggetto in props quando lo passi; la destrutturazione *spacchetta* le props quando le usi.

```
<Prodotto {...primaCard} />           // impacchetta  
const Prodotto = ({ nome, prezzo, img }) => {...} // spacchetta
```

Il Children Props

Definizione. `children` è una prop speciale e automatica: contiene tutto ciò che scrivi **tra il tag di apertura e quello di chiusura** di un componente.

```
// Componente contenitore  
const Card = ({ children }) => {  
  return <div className="card">{children}</div>;  
};  
  
// Utilizzo  
const App = () => (  
  <Card>  
    <h2>Titolo</h2>  
  </Card>  
)
```

```
<p>Un paragrafo qualsiasi</p>
<button>Azione</button>
</Card>
```

);

Tutto ciò che sta dentro `<Card> . . . </Card>` arriva al componente nella prop `children` e viene disegnato dove tu scrivi `{children}`.

Quando serve. È il pattern per i **componenti contenitore**: quelli che definiscono un aspetto o un comportamento ma non sanno in anticipo cosa conterranno. Modal, sidebar, layout, card, tooltip.

```
const Modal = ({ titolo, children }) => (
```

```
<div className="overlay">
  <div className="modal">
    <h2>{titolo}</h2>
    {children}
    <button>Chiudi</button>
  </div>
</div>
```

);

```
// Lo stesso Modal con contenuti completamente diversi
```

```
<Modal titolo="Conferma">
  <p>Vuoi davvero eliminare?</p>
</Modal>

<Modal titolo="Login">
  <input type="email" />
  <input type="password" />
</Modal>
```

Senza `children` dovresti passare il contenuto come stringa e perderesti la possibilità di inserire markup arbitrario.

Iterare un Array in JSX

Definizione. Trasformare un array di dati in un array di elementi JSX. Si fa con il metodo `map()`.

Perché proprio map. JSX accetta un array di elementi e li renderizza in sequenza. `map()` è l'unico metodo che *ritorna un nuovo array* trasformando ogni elemento — `forEach` non ritorna nulla, quindi non funziona.

```
const nomi = ["Anna", "Marco", "Giulia"];
```

```
const Lista = () => {  
  return (  
    <ul>  
      {nomi.map((nome) => (  
        <li key={nome}>{nome}</li>  
      ))}  
    </ul>  
  );  
};
```

Scomposto passo per passo:

1. `{` — entro in JavaScript
2. `nomi.map(...)` — trasformo ogni stringa in un `<li>`
3. ottengo [`<li>Anna</li>`, `<li>Marco</li>`, `<li>Giulia</li>`]
4. React disegna tutti gli elementi dell'array

Attenzione al return implicito. Usando le parentesi tonde, la arrow function ritorna automaticamente. Con le graffe devi scrivere `return` esplicitamente:

```
{nomi.map((nome) => {  
  const maiuscolo = nome.toUpperCase();  
  return <li key={nome}>{maiuscolo}</li>; // return obbligatorio  
})}
```

Dimenticarlo produce una lista vuota, ed è un errore molto comune.

Iterare un array di oggetti

Definizione. Il caso reale: i dati non sono mai stringhe semplici, ma oggetti provenienti da un'API o da un file.

```
const prodotti = [  
  { id: 1, nome: "Cuffie", prezzo: 49.9, img: "cuffie.jpg" },  
  { id: 2, nome: "Mouse", prezzo: 19.9, img: "mouse.jpg" },
```

```

    { id: 3, nome: "Tastiera", prezzo: 79.9, img: "tastiera.jpg" }
  ];

const ListaProdotti = () => {
  return (
    <section className="griglia">
      {prodotti.map((prodotto) => (
        <Prodotto key={prodotto.id} {...prodotto} />
      ))}
    </section>
  );
};

```

Questa riga è il punto di arrivo di tutta la Lezione 2. In una sola espressione convergono quattro concetti: `map` per iterare, `key` per l'identificazione, lo `spread` per passare le props, e il componente riutilizzabile che le riceve destrutturate.

Puoi anche destrutturare direttamente nel parametro del `map`:

```

{prodotti.map(({ id, nome, prezzo }) => (
  <li key={id}>{nome} — {prezzo} €</li>
))}

```

Il Key Attribute

Definizione. `key` è un attributo speciale, obbligatorio su ogni elemento generato dentro un `map`. Serve a React per identificare univocamente ogni elemento della lista tra un render e l'altro.

Perché esiste davvero. Torniamo al Virtual DOM. Quando la lista cambia, React confronta vecchia e nuova versione. Senza `key` può solo confrontare per posizione — e se inserisci un elemento in cima, React crede che *tutti* gli elementi siano cambiati e li ridisegna tutti. Con `key` riconosce che sono gli stessi elementi solo spostati, e muove i nodi invece di ricrearli.

Le regole della key:

1. **Deve essere unica tra fratelli** (non globalmente unica in tutta l'app).
2. **Deve essere stabile:** lo stesso elemento deve avere sempre la stessa key.
3. **Idealmente è l'id del dato**, che di norma arriva già dall'API o dal database.

```

{prodotti.map((p) => <Prodotto key={p.id} {...p} />)}

```

Perché l'indice è una cattiva idea:

```
{prodotti.map((p, index) => <Prodotto key={index} {...p} />)}
```

Se elimini il primo elemento, tutti gli indici scalano: l'elemento che prima era `key=1` diventa `key=0`. React pensa che il contenuto dell'elemento 0 sia cambiato, invece di capire che l'elemento 0 è stato rimosso. Con componenti che hanno uno stato interno (un input compilato, una checkbox spuntata) questo produce bug visibili: il testo scritto in un campo "salta" su un'altra riga.

Usare l'indice è accettabile solo se la lista è statica, non viene mai riordinata né filtrata, e gli elementi non hanno stato proprio.

Il messaggio di errore che vedrai in console se dimentichi la `key` è: *"Warning: Each child in a list should have a unique key prop"*. Non blocca l'applicazione, ma va sempre risolto.

Gestione degli eventi in JSX

Definizione. Gli eventi sono le azioni dell'utente (click, digitazione, invio di un form) a cui il componente reagisce. In JSX si gestiscono con attributi in camelCase che ricevono **una funzione**.

```
const Bottone = () => {  
  const gestisciClick = () => {  
    console.log("Cliccato!");  
  };  
  
  return <button onClick={gestisciClick}>Clicca qui</button>;  
};
```

L'errore numero uno in assoluto, quello su cui inciampano tutti:

```
<button onClick={gestisciClick}> // CORRETTO: passo la funzione  
<button onClick={gestisciClick()}> // SBAGLIATO: la ESEGUO subito
```

Nel secondo caso la funzione viene chiamata durante il render, non al click. Se quella funzione modifica lo stato, ottieni un loop infinito.

Se devi passare un argomento, avvolgi in una arrow function:

```
const elimina = (id) => console.log("Elimino", id);  
  
<button onClick={() => elimina(prodotto.id)}>Elimina</button>  
<button onClick={() => elimina(prodotto.id)}>Elimina</button>
```

Così `onClick` riceve una funzione anonima, e solo al click quella funzione chiamerà `elimina(id)`.

Gli eventi più usati:

Evento	Quando scatta
onClick	Click su un elemento
onChange	Il valore di un input cambia (ad ogni carattere)
onSubmit	Invio di un form
onmouseenter / onMouseleave	Il mouse entra/esce dall'elemento
onKeyDown	Pressione di un tasto

L'oggetto evento. React passa automaticamente alla funzione un oggetto evento (un *SyntheticEvent*, wrapper cross-browser sull'evento nativo):

```
const gestisciInput = (e) => {  
  console.log(e.target.value); // il testo digitato  
};
```

```
<input type="text" onChange={gestisciInput} />
```

`e.target` è l'elemento che ha generato l'evento, `e.target.value` il suo valore corrente.

preventDefault(). Fondamentale sui form, perché il comportamento predefinito di un form è ricaricare la pagina — cosa che in una SPA distrugge tutto lo stato:

```
const gestisciSubmit = (e) => {  
  e.preventDefault();  
  console.log("Form inviato senza ricaricare");  
};
```

```
<form onSubmit={gestisciSubmit}>  
  <input type="text" />  
  <button type="submit">Invia</button>  
</form>
```

Publicare progetti su Netlify

Definizione. Netlify è una piattaforma di hosting per siti statici. Il deploy è il processo che porta il progetto dal tuo computer a un indirizzo pubblico su internet.

Prima di tutto: la build.

```
npm run build
```

Questo comando genera la cartella `build/` con HTML, CSS e JavaScript minificati e ottimizzati. Il codice React con JSX non può essere caricato online così com'è: va prima compilato. La cartella `build` è ciò che si pubblica.

Metodo 1 — Drag and drop. Vai su Netlify, sezione Sites, e trascini la cartella `build` nell'area indicata. Dopo pochi secondi il sito è online su un URL del tipo `random-name-123.netlify.app`, rinominabile dalle impostazioni. È il metodo più immediato, ma ogni aggiornamento richiede di rifare build e ritrascinare.

Metodo 2 — Collegamento a GitHub (continuous deployment). Colleghi il repository GitHub a Netlify e configuri:

- Build command: `npm run build`
- Publish directory: `build` (con Vite sarebbe `dist`)

Da quel momento ogni `git push` sul branch principale fa ripartire la build su Netlify e aggiorna il sito da solo. È il flusso professionale.

Due problemi tipici del deploy React:

Le variabili d'ambiente. Le chiavi API stanno in un file `.env` locale, escluso da Git:

```
REACT_APP_API_KEY=abc123
```

Su Netlify vanno reinserite manualmente in *Site settings* → *Environment variables*, altrimenti il sito pubblicato non riesce a chiamare le API. Nota: con `create-react-app` il prefisso obbligatorio è `REACT_APP_`; con Vite è `VITE_`.

Il 404 sulle rotte. Se usi React Router e ricarichi la pagina su `/contatti`, ottieni un errore 404: il server cerca un file `contatti.html` che non esiste, perché in una SPA il routing è gestito da JavaScript. La soluzione è un file `public/_redirects` con una riga:

```
/* /index.html 200
```

Significa: qualunque URL richiesto, servi `index.html` e lascia che sia React Router a decidere cosa mostrare.